Annexes - Correction d'erreur quantique

Projet : Projet_Correction_Erreur_Quantique

Objectif : regrouper les graphes generes par les scripts Python du depot et les extraits de code associes, dans une structure coherente avec le rapport : erreur X sur un qubit, correction X a 3 qubits, correction Z a 3 qubits, puis script classique complementaire.

Annexe	Contenu	Fichier(s)
A	Erreur X apres une porte X	Simulation_erreur_porte_X_1_qubit.py - graph_2.png
B	Syndromes et succes de correction X	Simulation_erreur_porte_X_3_qubits.py - graph_3a/3b.png
C	Syndromes et succes de correction Z	Simulation_erreur_porte_Z_3_qubits.py - graph_4a/4b.png
D	Correction classique a 3 bits	Correction_Classique_3_bits.py

Annexe A - Erreur de porte X sur un qubit

Cette annexe correspond au graphe de comparaison entre les probabilites theoriques P(0)=p, P(1)=1-p et les frequences simulees apres une porte X bruitee.

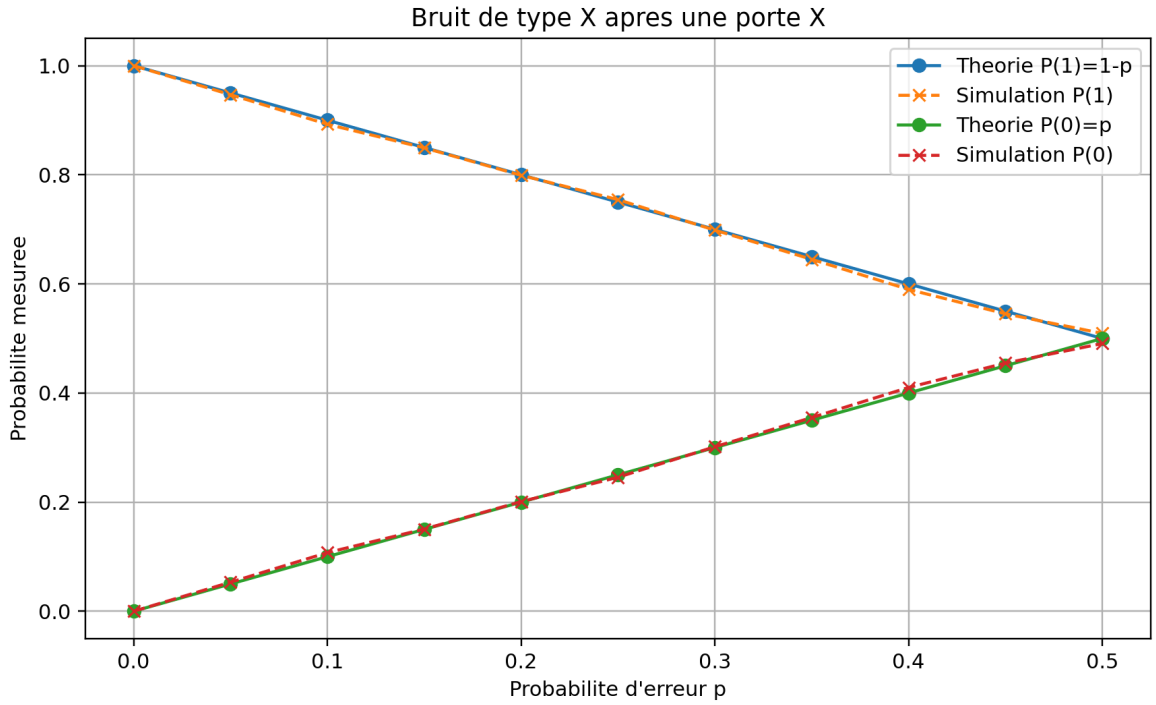

Figure A.1 - Simulation d'une erreur X apres une porte X : theorie contre simulation.

Code source associe

```
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from qiskit_aer.noise import NoiseModel, pauli_error
import numpy as np
import matplotlib.pyplot as plt

SHOTS = 4096
P_VALUES = np.linspace(0, 0.5, 11)
SEED = 12345

def build_bit_flip_noise_model(p):
    error_x = pauli_error([("I", 1 - p), ("X", p)])
    noise_model = NoiseModel()
    noise_model.add_all_qubit_quantum_error(error_x, ["x"])
    return noise_model

def x_gate_circuit():
    qc = QuantumCircuit(1, 1)
    qc.x(0)
    qc.measure(0, 0)
    return qc

def run_simulation(p):
    noise_model = build_bit_flip_noise_model(p)
    simulator = AerSimulator(noise_model=noise_model, seed_simulator=SEED)
    qc = x_gate_circuit()
    tqc = transpile(qc, simulator)
    result = simulator.run(tqc, shots=SHOTS).result()
    return result.get_counts()

def frequency(counts, bit):
    return counts.get(bit, 0) / SHOTS

theory_p0, theory_p1 = [], []
simulation_p0, simulation_p1 = [], []
for p in P_VALUES:
    counts = run_simulation(float(p))
    theory_p0.append(p)
    theory_p1.append(1 - p)
    simulation_p0.append(frequency(counts, "0"))
    simulation_p1.append(frequency(counts, "1"))

plt.figure(figsize=(8, 5))
plt.plot(P_VALUES, theory_p1, marker="o", label="Theorie P(1)=1-p")
plt.plot(P_VALUES, simulation_p1, marker="x", linestyle="--", label="Simulation Qiskit P(1)")
plt.plot(P_VALUES, theory_p0, marker="o", label="Theorie P(0)=p")
plt.plot(P_VALUES, simulation_p0, marker="x", linestyle="--", label="Simulation Qiskit P(0)")
plt.xlabel("Probabilite d'erreur p")
```

```
plt.ylabel("Probabilite mesuree")
plt.title("Bruit de type X apres une porte X")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.savefig("graph_2.png", dpi=300)
plt.show()
```

Annexe B - Correction des erreurs X avec le code a 3 qubits

Cette section regroupe les frequences des syndromes et le taux de succes de la correction pour un modele d'erreur bit-flip.

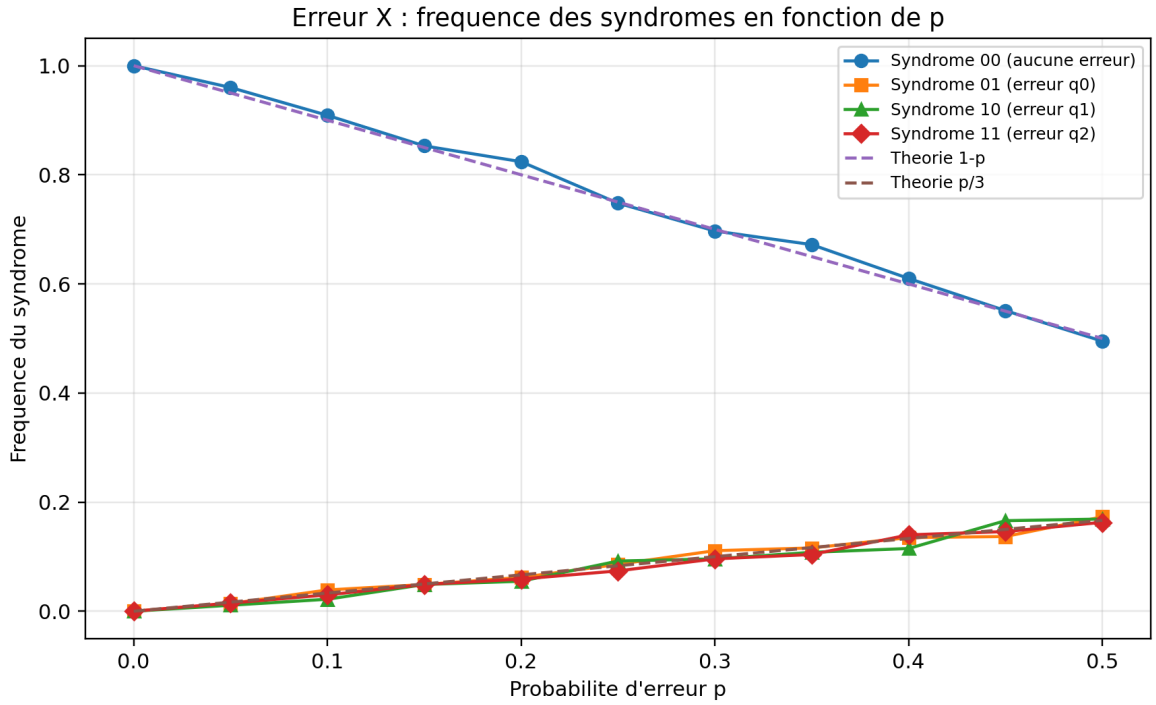

Figure B.1 - Frequence des syndromes pour les erreurs X.

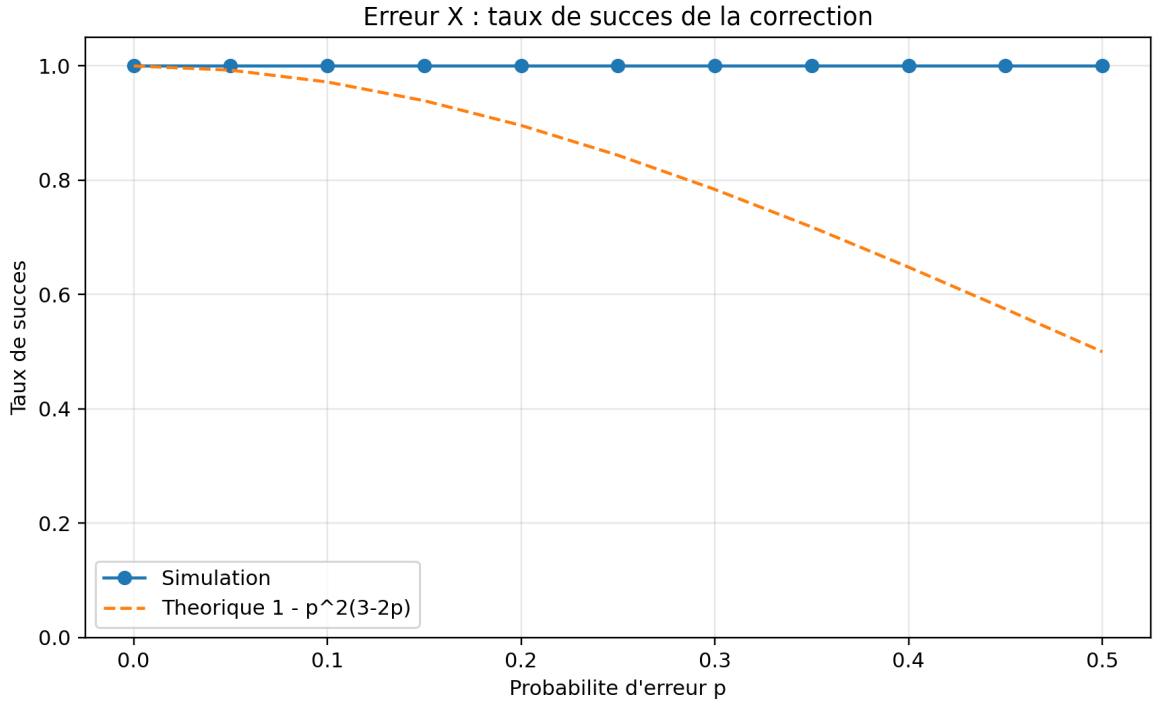

Figure B.2 - Taux de succes de la correction X.

Code source associe

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
import numpy as np
import matplotlib.pyplot as plt

sim = AerSimulator()
```

```
P_VALUES = np.linspace(0, 0.5, 11)
N_SHOTS = 500

def syndrome_block_X(qc):
    qc.cx(0, 3)
    qc.cx(1, 3)  # ancilla 3 = q0 XOR q1
    qc.cx(0, 4)
    qc.cx(2, 4)  # ancilla 4 = q0 XOR q2

def appliquer_correction(qc, syndrome):
    match syndrome:
        case '01': qc.x(0)
        case '10': qc.x(1)
        case '11': qc.x(2)

def simuler_un_tir_X(p):
    qc1 = QuantumCircuit(5, 2)
    qc1.h(0)
    qc1.cx(0, 1)
    qc1.cx(0, 2)
    qubit_erreur = None
    if np.random.rand() < p:
        qubit_erreur = int(np.random.rand() * 3)
        qc1.x(qubit_erreur)
    syndrome_block_X(qc1)
    qc1.measure(3, 1)
    qc1.measure(4, 0)
    result1 = sim.run(qc1, shots=1).result()
    s = list(result1.get_counts().keys())[0][-2:]

    qc2 = QuantumCircuit(5, 2)
    qc2.h(0)
    qc2.cx(0, 1)
    qc2.cx(0, 2)
    if qubit_erreur is not None:
        qc2.x(qubit_erreur)
    appliquer_correction(qc2, s)
    syndrome_block_X(qc2)
    qc2.measure(3, 1)
    qc2.measure(4, 0)
    result2 = sim.run(qc2, shots=1).result()
    s2 = list(result2.get_counts().keys())[0][-2:]
    return s, (s2 == '00')

def simuler_p_X(p, n_shots):
    freq = {'00': 0, '01': 0, '10': 0, '11': 0}
    succes = 0
    for _ in range(n_shots):
        syndrome, ok = simuler_un_tir_X(p)
        freq[syndrome] += 1
        if ok: succes += 1
    for k in freq: freq[k] /= n_shots
    return freq, succes / n_shots

resultats = {'p': [], 'f00': [], 'f01': [], 'f10': [], 'f11': [], 'succes': []}
for p in P_VALUES:
    freq, suc = simuler_p_X(p, N_SHOTS)
    resultats['p'].append(p)
    resultats['f00'].append(freq['00'])
    resultats['f01'].append(freq['01'])
    resultats['f10'].append(freq['10'])
    resultats['f11'].append(freq['11'])
    resultats['succes'].append(suc)

p_array = np.array(P_VALUES)
theo_p3 = p_array**2 * (3 - 2*p_array)
# puis: construction de graph_3a.png et graph_3b.png avec Matplotlib.
```

Annexe C - Correction des erreurs Z avec le code a 3 qubits

Cette section reprend la meme logique que l'annexe B dans la base de Hadamard, afin de traiter les erreurs de phase Z.

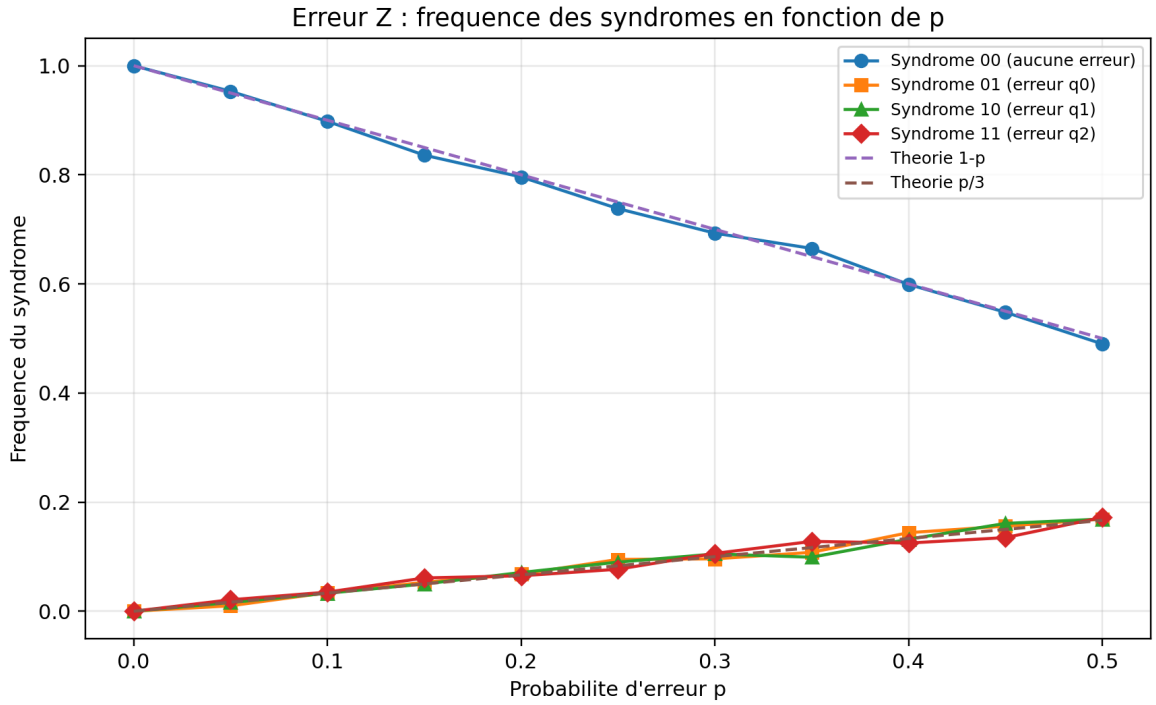

Figure C.1 - Frequence des syndromes pour les erreurs Z.

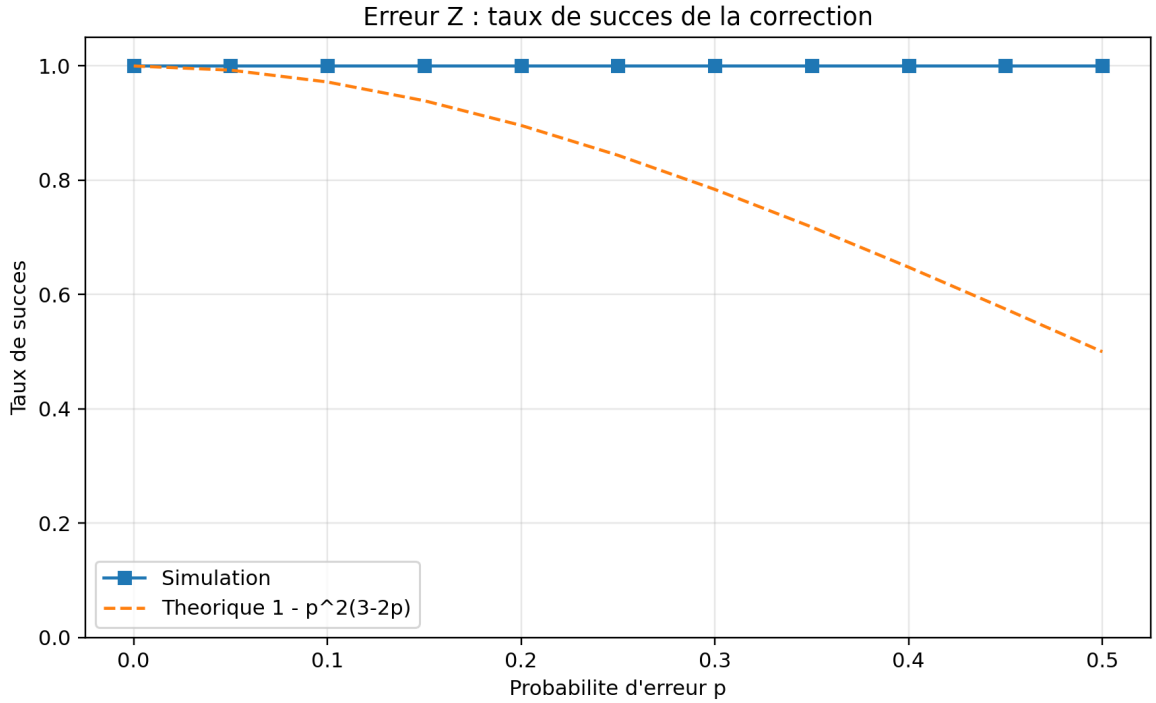

Figure C.2 - Taux de succes de la correction Z.

Code source associe

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
import numpy as np
import matplotlib.pyplot as plt

sim = AerSimulator()
```

```
P_VALUES = np.linspace(0, 0.5, 11)
N_SHOTS = 500

def syndrome_block_Z(qc):
    qc.h(0); qc.h(1); qc.h(2)
    qc.cx(0, 3)
    qc.cx(1, 3)
    qc.cx(0, 4)
    qc.cx(2, 4)
    qc.h(0); qc.h(1); qc.h(2)

def appliquer_correction_Z(qc, syndrome):
    match syndrome:
        case '01': qc.z(0)
        case '10': qc.z(1)
        case '11': qc.z(2)

def simuler_un_tir_Z(p):
    qc1 = QuantumCircuit(5, 2)
    qc1.h(0)
    qc1.cx(0, 1)
    qc1.cx(0, 2)
    qc1.h(0); qc1.h(1); qc1.h(2)
    qubit_erreur = None
    if np.random.rand() < p:
        qubit_erreur = int(np.random.rand() * 3)
        qc1.z(qubit_erreur)
    syndrome_block_Z(qc1)
    qc1.measure(3, 1)
    qc1.measure(4, 0)
    result1 = sim.run(qc1, shots=1).result()
    s = list(result1.get_counts().keys())[0][-2:]

    qc2 = QuantumCircuit(5, 2)
    qc2.h(0)
    qc2.cx(0, 1)
    qc2.cx(0, 2)
    qc2.h(0); qc2.h(1); qc2.h(2)
    if qubit_erreur is not None:
        qc2.z(qubit_erreur)
    appliquer_correction_Z(qc2, s)
    syndrome_block_Z(qc2)
    qc2.measure(3, 1)
    qc2.measure(4, 0)
    result2 = sim.run(qc2, shots=1).result()
    s2 = list(result2.get_counts().keys())[0][-2:]
    return s, (s2 == '00')

# La suite agrège les fréquences de syndromes puis exporte graph_4a.png et graph_4b.png.
```

Annexe D - Script classique complementaire

Le script suivant illustre le principe de repetition et de vote majoritaire sur trois bits, utile pour introduire l'idee de correction par redondance.

```
import numpy as np
import random as rd

q0 = np.array([1, 0])
q1 = np.array([0, 1])
q = (1 / (2**0.5)) * (q0 + q1)

X = np.array([[0, 1], [1, 0]])
Z = np.array([[1, 0], [0, -1]])

def bit_flip_erreur(q, p_bit):
    if rd.random() < p_bit:
        return X @ q
    return q

def phase_erreur(q, p_phase):
    if rd.random() < p_phase:
        return Z @ q
    return q

def encoder3bits(q):
    return [q, q, q]

def bit_flip(q, p):
    q3 = encoder3bits(q)
    for i in range(len(q3)):
        if rd.random() < p:
            q3[i] = 1 - q3[i]
    return q3

def corriger(q3):
    return 1 if sum(q3) >= 2 else 0

def simulation(p, Nmc):
    erreurs_sans_correction = 0
    erreurs_avec_correction = 0
    for _ in range(Nmc):
        bit_initial = rd.randint(0, 1)
        bit_bruite = bit_flip(bit_initial, p)[0]
        if bit_bruite != bit_initial:
            erreurs_sans_correction += 1
        code_bruite = bit_flip(bit_initial, p)
        bit_corrige = corriger(code_bruite)
        if bit_corrige != bit_initial:
            erreurs_avec_correction += 1
    return erreurs_sans_correction / Nmc, erreurs_avec_correction / Nmc
```

Remarque : les figures de ce PDF ont ete regenerees a partir de la logique des scripts du depot. Les simulations Qiskit completes necessitent les dependances qiskit et qiskit-aer dans l'environnement local.